Software Requirement Specification and Design

Multi Modal Smart Attendance System

Submitted By

S. No.	Team Member	Roll No.
1	Tejaswi Anand	24293916146
2	Prerna Priyadarshi	24293916077
3	Chhering Palmo	24293916107
4	Abhishek	24293916113

To be Submitted to

Dr. Rekha R

Department of Computer Science Engineering

Faculty of Technology

University of Delhi

Faculty of Technology
University of Delhi

New Delhi (2026-27)

Table of Contents

Table of Contents ii
Revision History ii

1.0 Introduction 04

1.1 Purpose 04
1.2 Scope of Project 04
1.3 Glossary 05
1.4 References 05
1.5 Overview of Document 06

2.0 Overall Description 06

2.1 System Environment 06
2.2 Functional Requirements Specification 07
2.2.1 User Use Cases 07
2.2.2 Admin Use Cases 07

2.3 User Characteristics 08
2.3.1 System Administrators (Admins) 08
2.3.2 Registered Users (Students/Employees) 08

2.4 Non Functional Requirements 09
2.4.1 Performance and Scalability 09
2.4.2 Reliability and Availability 09
2.4.3 Security and Privacy 09
2.4.4 Environmental and Hardware Constraints 10

3.0 Requirement Specification 11

3.1 External Interface Requirements 11

3.2 Functional Requirements 11
3.2.1 Enroll User 11
3.2.2 Capture Face Data 11
3.2.3 Generate Encoding 11
3.2.4 Monitor Live Stream 11

3.2.5 Detect Face 11
3.2.6 Verify Identity 12
3.2.7 Log Attendance 12
3.2.8 View Recent Activity 12
3.2.9 Manage Records 12
3.2.10 Analyze Attendance 12
3.2.11 Export Data 12

3.2.12 Authenticate Admin 12

3.3 Detailed Non Functional Requirements 13
3.3.1 Logical Structure of the Data 13
3.3.2 Security 13

3.3.2.1 Biometric Data Protection 13
3.3.2.2 Implementation Hierarchy 14
3.3.2.3 Encryption Protocols 14
3.3.2.4 Role Based Access Control (RBAC) 14
3.3.2.5 Database Security 14
3.3.2.6 Authentication 14
3.3.2.7 Network Deployment 14

4.0 Software Design 14

4.1 System Architecture 14
4.1.1 Presentation Layer 14
4.1.2 Application Layer 15
4.1.3 Data Layer 15
4.1.4 Hardware Interface Layer 15
4.1.5 Data Flow Overview 15

4.2 Detailed Design 16
4.2.1 Class Diagram 16

4.2.2 Sequence Diagrams 18

4.3 Database Schema 20

4.4 Security Design 21
4.4.1 Authentication and Authorization Mechanism 21

4.4.2 Security Protocols 21

5.0 Traceability 21

5.1 Requirement to Design Mapping 21

1.0. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) document provides a comprehensive technical overview of the Smart Attendance System (SAS). It details the architectural design, functional workflows, and technical constraints. The primary objective is to define a robust framework for an automated attendance solution that utilizes computer vision to minimize manual intervention and human error in tracking attendance.

1.2. Scope of Project

The SAS is an integrated desktop application designed to capture real-time video streams, perform facial detection, and verify identities against a secure database. The system is engineered for deployment in high-traffic environments like university lecture halls and corporate offices.

Key features include:

- Real-time multi-face detection and identification.
- Automated attendance logging with anti-spoofing considerations.
- Administrative dashboard for record management and analytics.
- Data portability through standardized export formats (CSV/XLSX).

1.3. Glossary

Term	Definition
OpenCV	Open-Source Computer Vision Library; the backbone for image manipulation and video I/O.
Dlib	A C++ toolkit containing machine learning algorithms; SAS utilizes its pre-trained facial landmark models.
Face Recognition Library	A Python wrapper for dlib's state-of-the-art face recognition built with deep learning.
Euclidean Distance	A mathematical measure of the straight-line distance between two points in 128-dimensional space.
HOG	Histogram of Oriented Gradients; a feature descriptor used for object detection.
CNN	Convolutional Neural Network. Used here for high-accuracy facial detection in complex environments.
AES-256	Advanced Encryption Standard with a 256-bit key size for securing biometric templates.

1.4. References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.
- ISO/IEC 19794: Biometric Data Interchange Formats.
- NIST Face Recognition Vendor Test (FRVT) Guidelines.
- Python 3.0+ Documentation

1.5. Overview of Document

This Software Requirements Specification (SRS) is organized into three major sections to provide a comprehensive technical roadmap for the Smart Attendance System (SAS). The structure is designed to guide stakeholders through the project's objectives, technical architecture, and implementation details.

- Section 1: Introduction defines the primary objectives of the SAS, provides a glossary of technical terms (such as OpenCV, HOG, and CNN), and outlines the references used in the development process.
- Section 2: Overall Description establishes the system’s perspective, including the hardware interface layer, processing environment, and detailed operational workflows for both User Registration and Attendance Recognition.
- Section 3: Requirements Specification details the functional requirements, the PyQt6-based graphical user interface (GUI) design, and the data management capabilities of the system.

2.0. Overall Description

2.1. System Environment

The SAS operates as a unified software package. It interfaces with local imaging hardware (Standard CMOS Webcams or IP-based RTSP Cameras) and a local or networked SQL database.

- Imaging Layer: Captures raw frames (720p/1080p) at 30 FPS.
- Processing Layer: Python 3.10+ execution environment leveraging CUDA-enabled libraries (optional) for accelerated inference.
- Data Layer: SQLite for local deployment or PostgreSQL for enterprise-scale requirements.

2.2. Functional Requirements Specification

2.2.1 User Use Cases

Use case: Automatic Attendance Logging

- Description: The system automatically captures live video frames to detect and identify registered users.
- Workflow: The system performs real-time face detection, calculates Euclidean distance against stored encodings, and logs attendance once identity is stable for three consecutive frames.
- Output: A "Success" indicator is displayed on the GUI, and a record is added to the Attendance_Logs table.

2.2.2 Admin Use Cases

Use case: User Enrollment

- Description: The Administrator registers new individuals into the secure database.
- Workflow: Admin enters metadata (ID, Name, Dept), initiates a 5-second video burst for image acquisition, and the system generates a 128-dimensional encrypted encoding.

Use case: Administrative Dashboard Management

- Description: Accessing the central control panel to manage system settings and records.
- Workflow: Admin interacts with the PyQt6-based interface to monitor recent activity and adjust administrative settings.

Use case: Data Export

- Description: Generating portable attendance records for external use.
- Workflow: The Admin uses a one-click functionality to export logs into standardized .csv or .xlsx formats following ISO 8601 date standards.

Use case: Attendance Analysis

- Description: Reviewing filtered data to monitor attendance trends or requirements.

- Workflow: Using the Data Manager and Pandas integration, the Admin can generate reports for specific criteria, such as identifying students with less than 75% attendance.

2.3. User Characteristics

This section outlines the primary users of the Smart Attendance System (SAS), their roles, and the technical expertise required to operate or interact with the system effectively. The SAS is designed to function in high-traffic environments such as university lecture halls and corporate offices.

2.3.1 System Administrators (Admins)

Administrators are the primary power users of the system, responsible for maintaining the integrity of the attendance database and managing the hardware interface.

- Responsibilities: They handle the enrollment of new individuals, which involves entering unique metadata (ID, name, and department) and overseeing the biometric capture process.
- System Interaction: Admins use the PyQt6-based graphical interface to monitor real-time activity, manage system settings, and perform data maintenance.
- Technical Proficiency: They are expected to have a moderate level of technical knowledge, particularly in managing data exports to .csv and .xlsx formats and interpreting attendance analytics generated through the integrated Pandas library.
- Authorization: Admins hold elevated privileges under the Role-Based Access Control (RBAC) model, allowing them to access the secure administrative dashboard and encryption key management.

2.3.2 Registered Users (Students/Employees)

Registered users are the subjects of the attendance tracking process. Their interaction with the software is designed to be largely passive to ensure maximum efficiency.

- Responsibilities: Their primary duty is to be present within the camera's field of view during operational hours.
- Enrollment: During an initial one-time registration session, they must provide various facial angles to allow the system to compute a unique 128-dimensional floating-point encoding.
- Technical Proficiency: No technical expertise is required, as the system is engineered to minimize manual intervention and human error through automated computer vision workflows.

- Privacy Awareness: Users interact with a system that prioritizes privacy by ensuring no raw biometric images are stored permanently in the database.

2.4. Non-Functional Requirements

While functional requirements define what the system does, these non-functional requirements (NFRs) define how the system performs under various conditions. Think of these as the "behind-the-scenes" standards that ensure the Smart Attendance System (SAS) is reliable, secure, and fast enough for real-world use.

2.4.1 Performance and Scalability

- Real-time Processing: To maintain smooth operation, the system must achieve a processing speed of 20+ FPS.
- Responsiveness: The frame buffer pulls a new frame every 150ms to ensure the GUI remains responsive and real-time identification feels instantaneous.
- Inference Speed: The system utilizes downsampling (resizing frames to 25% of original resolution) specifically to boost detection speed without sacrificing significant accuracy.
- Modular Architecture: The system follows a layered pattern to ensure it can scale from a local desktop application to a networked enterprise environment.

2.4.2 Reliability and Availability

- System Uptime: The SAS is required to be operational 99.5% of the time during standard business hours.
- Recognition Accuracy: The underlying ResNet-34 model provides a 99.38% accuracy rate, ensuring enterprise-grade reliability in identification.
- Error Tolerance: The False Acceptance Rate (FAR)—the chance of incorrectly identifying a stranger as a registered user—must remain below 0.01%.

2.4.3 Security and Privacy

- Biometric Encryption: All 128-dimensional facial encodings are encrypted using AES-256 before being committed to the database.
- Privacy-First Design: To protect user privacy, the system does not store raw images; once an encoding is generated, the source image is purged from memory.

- Data Integrity: The system mandates SQL injection prevention through the use of parameterized queries in all database interactions.
- Access Control: Access to administrative features and sensitive data is restricted through Role-Based Access Control (RBAC).

2.4.4 Environmental and Hardware Constraints

- Lighting Requirements: For optimal facial feature extraction, the environment requires a minimum ambient light level of 300 lux.
- Hardware Acceleration: To meet performance targets, the host machine should ideally possess a CPU with AVX (Advanced Vector Extensions) or a CUDA-capable GPU.

3.0. Requirements Specification

3.1. External Interface Requirements

- User Interface (GUI): A clean, dark-themed interface built using PyQt6. It must feature a primary video viewport, a "Recent Activity" sidebar, and an Admin settings panel.
- Data Export: One-click export functionality to .csv and .xlsx following the ISO 8601 date format.

3.2. Functional Requirements

This section details the specific functional capabilities of the Smart Attendance System (SAS). Following the modular architecture of the system, these requirements cover enrollment, real-time processing, data management, and security.

3.2.1 Enroll User

The system shall allow an Administrator to input metadata for new users, including a unique user_id, full name, and department. This process initiates the identity creation workflow within the Data Layer.

3.2.2 Capture Face Data

Upon initiating enrollment, the system shall trigger a 5-second video burst via the Hardware Interface Layer to capture various facial angles of the subject.

3.2.3 Generate Encoding

The Recognition Engine shall select the clearest captured frame, detect the face, and compute a 128-dimensional floating-point encoding. The system must verify that this encoding does not already exist in the database to prevent duplicate registrations.

3.2.4 Monitor Live Stream

The Presentation Layer shall provide a real-time video viewport within a dark-themed PyQt6 GUI. This feed must be pulled every 150ms and downsampled to 25% resolution to ensure responsiveness.

3.2.5 Detect Face

The system shall utilize the Histogram of Oriented Gradients (HOG) algorithm to locate face bounding boxes within the live video frames.

3.2.6 Verify Identity

The Recognition Engine shall calculate the Euclidean distance between the live facial encoding and all stored encodings in the database. An identity is considered a match if the distance is below the threshold of 0.6.

3.2.7 Log Attendance

The system shall automatically log attendance only after an identity remains stable for three consecutive frames. The log entry must include a unique log_id, the user's ID, a precise timestamp, and the matching confidence score.

3.2.8 View Recent Activity

The GUI shall feature a "Recent Activity" sidebar that provides visual confirmation of successful attendance marks to the operator.

3.2.9 Manage Records

The Data Manager shall support full CRUD (Create, Read, Update, Delete) operations for user records and attendance logs. This includes the ability to retrieve stored encodings for comparison.

3.2.10 Analyze Attendance

Using the integrated Pandas library, the system shall allow Administrators to perform complex data manipulation, such as identifying individuals with an attendance rate below 75%.

3.2.11 Export Data

The system shall provide a one-click export functionality to generate reports in .csv or .xlsx formats. All exported dates must adhere to the ISO 8601 standard.

3.2.12 Authenticate Admin

To ensure secure operations, the system shall implement Role-Based Access Control (RBAC). The software must require a secure login, verifying hashed passwords against the database before granting access to administrative tools.

3.3. Detailed Non-Functional Requirements

This section provides the technical specifications and constraints that ensure the Smart Attendance System (SAS) operates securely and maintains high data integrity.

3.3.1 Logical Structure of the data

The system utilizes a relational database structure designed for local deployment using SQLite or enterprise-scale deployment using PostgreSQL. The data is organized into two primary tables to ensure modularity and efficient retrieval.

- Users Table: This serves as the primary registry for individuals. It stores unique identifiers, department details, and the encrypted 128-dimensional facial encoding .
- Attendance_Logs Table: This table tracks every successful identification event. It captures a unique log ID, the linked user ID, a precise timestamp, and a matching confidence score .
- Data Relationship: The architecture maintains a one-to-many relationship between the User and AttendanceLog entities, allowing the system to maintain a complete history for each registered individual.
- Privacy-First Design: A critical logical constraint is that the system does not store raw images. Once the mathematical encoding is generated, the source image is immediately purged from memory to ensure user privacy.

3.3.2 Security

The SAS implements a multi-layered security framework to protect sensitive biometric data and restrict system access.

3.3.2.1 Biometric Data Protection

The system does not store raw images of users. Once the 128-d encoding is generated, the image is purged from memory.

- Encryption: The cryptography.fernet module is used for symmetric encryption. The key is stored in a secure local .env file or a system environment variable.
- Database Guard: SQL injection prevention is mandatory through parameterized queries in the Python database driver (sqlite3 or psycopg2).

3.3.2.2 Implementation Hierarchy

1. Capture Class: Manages threaded cv2.VideoCapture to prevent GUI freezing.
2. Recognition Engine: A singleton class managing the Dlib model loading and comparison logic.
3. Data Manager: Handles all CRUD operations and report generation via Pandas.

3.3.2.3 Encryption Protocols

Facial encodings are secured using AES-256 symmetric encryption. The system utilizes the cryptography.fernet module, with encryption keys stored securely in system environment variables rather than the source code.

3.3.2.4 Role-Based Access Control (RBAC)

The system distinguishes between standard users and Administrators. Only Admin users possess the authorization to register new individuals or access the administrative dashboard.

3.3.2.5 Database Security

To prevent unauthorized data manipulation, SQL injection protection is mandatory. All interactions with the database layer must use parameterized queries through the Python database driver.

3.3.2.6 Authentication

Access to the software requires credential verification where passwords are securely hashed before storage in the Users table .

3.3.2.7 Network Deployment

If deployed across a network, the use of the HTTPS protocol is strongly recommended to maintain data confidentiality during transmission.

4.0 Software Design

4.1 System Architecture

The Smart Attendance System follows a layered architectural pattern to ensure modularity, scalability, and separation of concerns. The system is divided into four primary layers:

4.1.1 Presentation Layer

This layer consists of the PyQt6-based graphical user interface (GUI). It provides:

- Real-time video viewport
- Administrative controls
- Attendance status indicators
- Report generation interface

The Presentation Layer handles user interaction and displays system outputs.

4.1.2 Application Layer

This layer contains the core processing logic of the system. It includes:

- Face detection module
- Facial encoding generator
- Recognition engine
- Decision logic for attendance confirmation

The Application Layer processes captured frames and performs identity verification.

4.1.3 Data Layer

This layer manages persistent storage and retrieval of system data. It includes:

- User records
- Encrypted facial encodings
- Attendance logs
- Reporting queries

The Data Layer ensures secure storage and efficient data management.

4.1.4 Hardware Interface Layer

This layer manages interaction with input devices such as webcams or IP cameras. It captures raw image frames and passes them to the Application Layer for processing.

4.1.5 Data Flow Overview

The overall data flow is as follows:

Camera → Capture Module → Recognition Engine → Data Manager → Database → GUI Update

This structured architecture improves maintainability and allows independent modification of components.

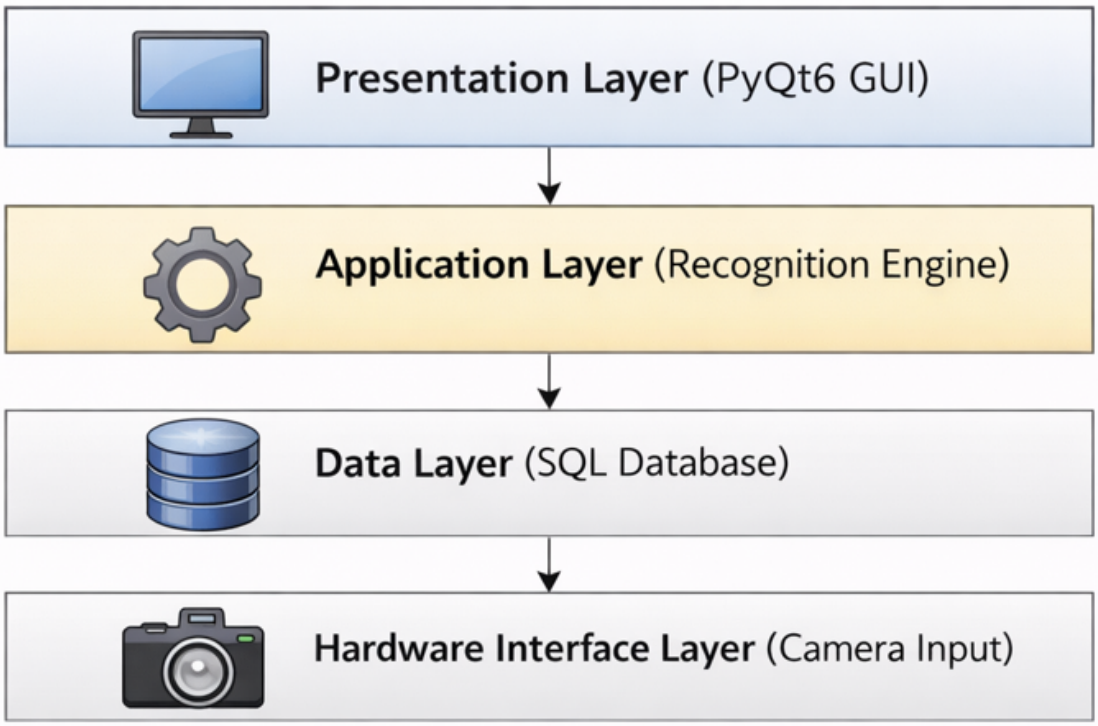


Figure 4.1: System Architecture Diagram

4.2 Detailed Design

4.2.1 Class Diagram

The system is designed using object-oriented principles. The primary classes are described below:

Capture

Responsibilities:

- Initialize camera feed
- Start and stop video stream
- Capture image frames

Key Methods:

- start_stream()
- stop_stream()
- capture_frame()

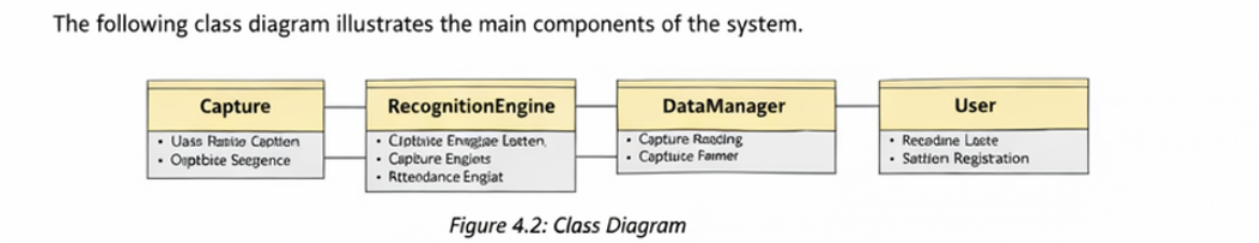


Figure 4.2: Class Diagram

RecognitionEngine

Responsibilities:

- Detect faces in captured frames
- Generate 128-dimensional facial encodings
- Compare live encoding with stored encodings

Key Methods:

- detect_face()
- generate_encoding()
- compare_encoding()

DataManager

Responsibilities:

- Store user information
- Log attendance records
- Export attendance data
- Retrieve stored encodings

Key Methods:

- save_user()
- log_attendance()
- export_data()

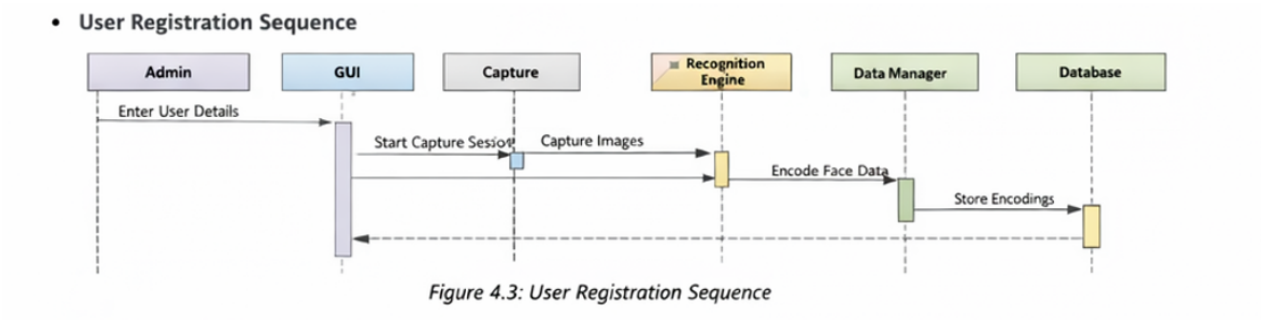


Figure 4.3: User Registration Sequence

User

Attributes:

- user_id
- name
- department
- encoding

Attendance Log

Attributes:

- log_id
- user_id
- timestamp
- confidence

Relationships

- RecognitionEngine depends on Capture for frame input.
- DataManager interacts with the Database for persistent storage.
- User and AttendanceLog maintain a one-to-many relationship.

4.2.2 Sequence Diagrams

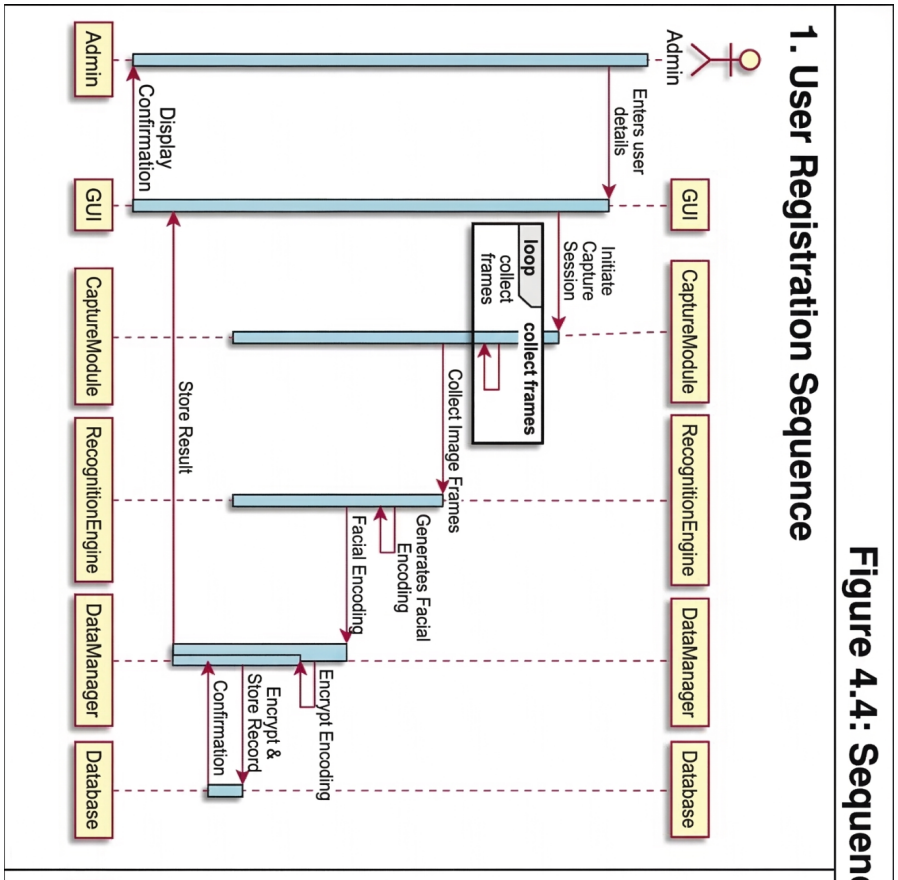
Two major operational sequences are defined.

1. User Registration Sequence

1. Admin enters user details via GUI.
2. GUI initiates capture session.
3. Capture module collects image frames.
4. RecognitionEngine generates facial encoding.
5. DataManager encrypts encoding.
6. Database stores encrypted record.
7. GUI displays confirmation.

2. Attendance Logging Sequence

1. Camera captures live frame.
2. RecognitionEngine detects and encodes face.
3. Encoding is compared with stored records.
4. If similarity threshold is satisfied for three consecutive frames, identity is confirmed.
5. DataManager logs attendance entry.
6. Database updates attendance record.
7. GUI displays success message.



4.3 Database Schema

The system database consists of two primary tables.

Table: Users

Column	Type	Constraints	Description
user_id	VARCHAR(20)	PRIMARY KEY	Unique user identifier
name	VARCHAR(100)	NOT NULL	User name
dept	VARCHAR(50)	NOT NULL	Department
encoding	BLOB	NOT NULL	Encrypted facial encoding
reg_date	TIMESTAMP	DEFAULT CURRENT_TIME	Registration timestamp
pwd_hash	VARCHAR(255)	NOT NULL	Hash of User's password
role	VARCHAR(20)	NOT NULL	User's role

Table: Attendance_Logs

Column	Type	Constraints	Description
log_id	BIGINT	PRIMARY KEY AUTOINCREMENT	Unique log ID
user_id	VARCHAR(20)	FOREIGN KEY REFERENCES Users	Linked user
time_in	DATETIME	NOT NULL	Attendance time
confidence	FLOAT	CHECK (confidence BETWEEN 0-1)	Matching score

4.4 Security Design

4.4.1 Authentication and Authorization Mechanisms

The system implements role-based access control (RBAC).

- Only Admin users can register new individuals.
- System login requires credential verification.
- Passwords are securely hashed before storage.
- Access to database operations is restricted based on user role.

4.4.2 Security Protocols

The following security protocols are implemented:

- AES-256 encryption for facial encodings
- Fernet symmetric encryption module
- Parameterized SQL queries to prevent SQL injection
- Secure storage of encryption keys in environment variables
- No raw biometric images stored in database
- HTTPS protocol recommended for network deployment

These measures ensure data confidentiality, integrity, and protection of biometric information.

5.0 Traceability

5.1 Requirement-to-Design Mapping

The following table establishes traceability between system requirements and design artifacts.

Requirement / Use Case ID	**Requirement / Use Case Name**	**Design ID(s)**	**Design Artifact**
FR-01	User Registration	D-CL-01, D-	Class Diagram (`DataManager`),

Requirement / Use Case ID	Requirement / Use Case Name	Design ID(s)	Design Artifact
		SQ-01	Sequence Diagram
FR-02	Face Recognition	D-CL-02, D-SQ-02	Class Diagram (`RecognitionEngine`), Sequence Diagram
FR-03	Log Attendance	D-CL-03, D-SQ-02	Class Diagram (`DataManager`), Sequence Diagram
SR-01	AES-256 Biometric Encryption	D-CL-01, D-SEC-01	Class Diagram (`DataManager`), Security Design
SR-02	SQL Injection Protection	D-CL-01, D-SEC-01	Class Diagram (`DataManager`), Security Design
SR-03	Privacy Protection (No Raw Image Storage)	D-CL-02, D-SEC-01	Class Diagram (`RecognitionEngine`), Security Design
NFR-03	Stability Logic (3-Frame Confirmation)	D-CL-02, D-SQ-02	Class Diagram (`RecognitionEngine`), Sequence Diagram
ENV-01	Lighting Constraint (Minimum 300 lux)	D-HW-01	Hardware Interface Layer

This mapping ensures every requirement is supported by a corresponding design element, enabling complete traceability throughout the development lifecycle.